

FOR THE CRSSD ADMIN TEAM · JUNE 2026

Admin Walkthrough

Everything you need to set up Menu Hub for a new client, keep the design system in good shape, run the Figma pipeline, and stay unstuck when something breaks. Pairs with the User Walkthrough (which is what you hand to editors).

App URL

fcmenus.netlify.app

Supabase project

fc-events
(wysvknamfxtbehwetjxf)

Repo

[seanmarshall21/fc-menu-hub](https://github.com/seanmarshall21/fc-menu-hub)

Plugin

figma.com/community/plugin/1645758553039246791

01 Roles & access model

Every user has one of four roles, set on the `user_profiles` table:

ROLE	CAN DO	CANNOT DO
Admin	Everything — including Styles, Templates, Admin page, role changes, deletion of menus/events/series	—
Internal	Edit menus, manage sponsors at every level, approve edits, view Approvals and Inbox	Edit Styles / Templates, manage users, delete events
External	Edit items (saves as Pending Approval), view Preview & Edit Log, can be tagged for notifications	Approve their own edits, edit sponsors, manage anything above the menu level
Pending	Sign in but only sees the "Access Pending" screen	Anything else — until admin assigns a real role

Roles are enforced by Postgres Row Level Security policies. Admins can read all profiles; internals can read other internal/admin profiles (via the `list_taggable_users()` RPC); externals only see their own row.

02 User management

Inviting a user

1. Sidebar → **Admin**
2. **Add User** (top right)
3. Email + name + role + (for external) brand access
4. Supabase sends them a magic-link invite email

Approving a pending signup

If someone signs up themselves (via the login page's signup form), they land in the user list with role = `pending` and a yellow row. Click **Approve** on the row to assign a real role, or **Deny** to delete.

Resetting a password

Each user row in Admin has a key icon (left of the pencil). Click it → confirm → Supabase emails the user a password-reset link that redirects to `/reset-password`.

Removing a user

Click the trash icon in an Admin user row. Deletes the `user_profiles` row. Their Supabase Auth user remains until removed from the Supabase dashboard — that's intentional so any historical `edit_log` rows still resolve to a real email.

03 Brands · series · events

Creating a brand

Sidebar → + next to "BRANDS" → fill in name, slug (auto-generated), color, icon. Admin-only.

Creating a series

Open the brand → + **New Series** (top right of Series list). Name + slug.

Creating an event

Open the series → + **New Event**. Name, slug, date, venue, phase, optional Figma file URL + Figma page name.

The phase pill

PHASE	MEANING
Build	In progress — content being assembled
Proof	Ready for review
Approved	Signed off — content frozen, edits return to Pending Approval

Duplicating

See section 14 — every level (menu / event / series) has a  dropdown with a Duplicate option that cascades through children.

Deleting



Deletes cascade.

Deleting an event removes every menu, every item, every sponsor toggle, every edit log row, every template under it. There's no undo. The typed-DELETE confirmation modal exists for a reason.

04 Series · Event · Menu Styles

Three-tier inheritance: Series sets the brand defaults; Event overrides per-event; Menu overrides per-menu.

TAB	WHERE	CONTROLS
Series Styles	Series page → Styles (admin-only)	The full design spec: fonts, type roles (menu title, item title, description, etc.), spacing (per size — SM / MD / LG), dietary icons, sponsor styling, footer config
Event Styles	Event page → Styles (admin-only)	Same fields as Series, but each is an inherit/override toggle. Default = inherit from series
Menu Styles	Menu page → Styles (admin-only)	Same fields, three-tier inheritance (series → event → menu). Useful for one-off menu tweaks

Spacing — auto + multiplier modes

- **Auto:** gap distributes leftover space via flexbox. Set a **min** floor so it never collapses below that
- **Section gap = Item gap × multiplier:** when on, section gap is computed from the resolved item gap (including auto-distributed values) × the multiplier. The renderer measures actual content heights, computes the item gap from available space, and derives section gap

Tips

- Per-size tabs (SM / MD / LG) under the Spacing block — each menu size has its own gap spec
- Item Title → Description gap is surfaced in the main spacing grid. Pricing-related fields (Content → Price, stacked/row gap, layout selects) live in the "Item Pricing" subgroup
- Reset columns (above the items table) restores default column order/widths

05 Event templates

The Templates tab on an Event stores the per-size background image + a color/style snapshot used by the in-app preview canvas and the Figma plugin.

Setting up

1. Event → **Templates**
2. For each size you'll use (SM / MD / LG), upload a background image OR push one from Figma via the plugin's Template Export tab
3. Set colors (section, title, description, price, size label, divider) — these drive the in-app preview

Pushing layout from Figma

In the Menu Sync plugin → Template Export tab → select size → "Push background + layout". The plugin uploads a JPEG of the frame's background and extracts spec tokens (gaps, padding, fonts) back into the event template.

06 Sponsor library & SVG normalization

Sponsors live in a global library (`sponsors` table) and cascade down:

Library → **Series** (which library entries this series uses) → **Event** (which series entries are active for this event) → **Menu** (which event entries appear on this specific menu)

Adding a sponsor to the library

1. Sidebar → **Sponsors**
2. **New Sponsor**
3. Name, slug (used for the Figma layer name `sponsor--slug`), Figma layer name override (optional), SVG upload, scale, max width

SVG auto-normalize on upload

When you upload an SVG, the app rewrites it before storing:

- Every hard-coded `fill="#..."` and `stroke="#..."` becomes `currentColor` (so the sponsor color override at series/event/menu level actually applies)
- Same for inline `style="fill:#..."`
- Root `<svg>` gets width/height stripped + `preserveAspectRatio="xMidYMid meet"` added so it scales freely
- `fill="none"` stays as `none` (intentional empties)

Internal users can upload too — the auto-normalize means they don't need to manually prep SVGs.

Sponsor approval (per menu)

Menu edit form → "Requires sponsor approval" toggle. When on, an amber chip appears at the top of that menu until someone clicks "Mark as approved". See section 8.

07 Approvals workflow

Status of an item drives where it appears + who can change it:

STATUS	VISIBLE IN	SET BY
active	Everywhere (table, preview, Figma sync, exports)	Default for new items + after approval
pending_approval	Items table (amber row), Edit Log	Auto: when anyone edits an Active item
not_added	Items table only (greyed)	Explicit "hide this item" toggle
draft	Items table only	Work-in-progress, default on bulk imports

Approving

- Per-item: Items table → row action → **Approve**. Or Edit Log → **Approve** on a row
- Per-menu: "Edits N pending" chip at top of menu → **Approve all**

Rejecting

Reject reverts the item to its pre-edit values (walks every unarchived edit_log row for the item, takes the earliest `old_value` per field). The log entries get `[rejected]` appended to their note for history.

Edit log accordions

Edit Log tab splits into Pending (open by default) / Approved / Rejected / History / Archived (all collapsed). Click chevrons to expand. Archive any row to hide it from active buckets.

08 Notify-for-edits cascade

People tagged on an edit get a notification in their Inbox. Defaults cascade top-down:

Brand → Series → Event → Menu → pre-checked on every item edit form

Setting at each level

WHERE	INHERITS FROM
Brand page → Approvals tab	(root — no parent)
Series → Approvals tab	Brand
Event → Approvals tab (below sign-off list)	Brand + Series
Menu → Approvals tab (below sign-off list)	Brand + Series + Event
Per-edit chips in item edit form	Brand + Series + Event + Menu (pre-checked, removable for this single edit)

Inherited people show as grayed chips with a check — can't be removed at a lower level (only at the level they were added). Protects against accidental exclusion.

Edit-resolved auto-notifications

When you approve or reject an edit, the original editor gets an auto-notification ("Your edit on X was approved"). Fires via a DB trigger on every `menu_items.edit_status` change. No frontend code involved — survives any UI changes.

Inbox

Sidebar → **Inbox** (red badge for unread count). Buckets: Tagged in edits · My edits · Archived (collapsed). Mark all read, archive all, restore archived, per-row actions.

09 Per-menu sponsor approval

Optional flag for menus where you need explicit sign-off from a sponsor (e.g. the Bulleit-sponsored craft cocktails menu where Bulleit's marketing team needs to OK the final).

Turning it on

1. Menu page → **Edit** button → tick "Requires sponsor approval"
2. Amber "Needs sponsor approval" chip appears at the top of that menu
3. When the sponsor signs off, click **Mark as approved** on the chip — flips to green "Approved by sponsor (timestamp)"

State stored on `menus.requires_sponsor_approval` , `menus.sponsor_approved_at` , `menus.sponsor_approved_by` .

10 Menu Sync plugin

The Figma plugin that pulls menu content from Menu Hub into your Figma menu templates.

Architecture: brand-prefixed masters, bare layers everywhere else

One Figma file holds every brand's masters and every event's pages. The plugin disambiguates which brand's masters to use via the brand's **Figma component prefix** (set in Edit Brand → defaults to the brand slug).

Master components (the source of truth — one set per brand, lives wherever you organize them in the file):

```
crssd--menu-item_layout_main      ← brand prefix on the master
crssd--menu-item_layout_alt
crssd--menu-items_section
utbs--menu-item_layout_main      ← UTBS has its own masters
utbs--menu-item_layout_alt
utbs--menu-items_section
```

Sponsor logos — prefix only if the styling is brand-specific. Shared logos can stay bare:

```
crssd--sponsor--bulleit          ← if Bulleit needs brand-specific style
sponsor--bulleit                 ← if the same Bulleit logo is reused across brands
```

Inside each event page — all layer names are bare. Duplicate the page, edit content, sync. No renaming:

```
menu-template_med_2x3           ← root frame (matches _sm _lg or small/med/large)
  menu_head_md
  menu-items_column
    menu-items_section
      section_label
    menu-items_section-items
      menu-item_layout_main      ← instance of the brand's master
      menu-item_details
      menu-item_price-group-column
        menu-item_price_group
        menu-item_size
        menu-item_price
        menu-item_price_group    ← second price (two-size items)
  menu_footer
    sponsors-row
      sponsor--bulleit          ← keep the sponsor-- prefix; brand prefix optional
```

Setting the brand's prefix

Brand page → **Edit Brand** → **Figma component prefix** field. Defaults to the brand slug — change only if your master components use something different (legacy `cf26f` instead of `crssd`, etc.).

Setting up a new brand

1. Create the brand in Menu Hub. The slug becomes the default prefix
2. In Figma, create the brand's master components: `<slug>--menu-item_layout_main` , `<slug>--menu-item_layout_alt` , `<slug>--menu-items_section` (one time per brand, ~2 min)
3. Build a template page using instances of those masters with bare layer names
4. Sync — works immediately. Every future event under that brand just duplicates the page

Setting up a new event under an existing brand

1. Right-click the most recent event's page in Figma → Duplicate page
2. Rename the page (e.g. UTBS 26 → UTBS 27)
3. Update the event's `Figma Page Name` in Menu Hub to match
4. Sync. No layer renaming, ever

Legacy templates with prefixed layers still work

If you have older templates where every layer is prefixed (`cf26f--menu-item_title` , etc.), the plugin still handles them — every name lookup OR's the prefixed and unprefixed forms. You can migrate them on your own schedule by editing the file in Figma; you don't have to do it in one go.

Sync flow

1. Plugin opens → cascading Brand → Series → Event → Menu dropdowns
2. "Fetch Menu Data" → loads items, sponsors, the menu's content fingerprint
3. Plugin auto-picks the matching size frame on the current page (warns if size doesn't match)
4. Toggle sponsors (pre-checked from Menu Hub's active set), footer options, spacing
5. "Sync to {frame name}" — rebuilds the items column from the master component instances

Frame ownership

Each synced frame stores `linkedMenuId` in its `pluginData`. The plugin never auto-picks a frame that's already linked to a different menu. Drop-down marks them "In use by other menus — pick only to overwrite" so you don't accidentally clobber spicy-pie when syncing main-bar.

Drift detection

On sync, the plugin writes a content fingerprint to the frame. On every fetch, it recomputes from current DB data. Mismatch → amber "Out of date" badge below the frame name. Match → green "Up to date with Menu Hub (synced X minutes ago)".

Update text only mode

Checkbox in the sync panel. When on, the plugin walks existing items in the column, matches by title, and only updates text content — preserving any manual layout tweaks you made in Figma.

Refresh preview image only

Button below Sync. Re-exports the frame as a PNG and uploads it to Menu Hub as the new preview thumbnail — without touching item text or marking the menu as freshly synced (drift indicator stays accurate).

11 Plugin distribution

Today: published on Figma Community

figma.com/community/plugin/1645758553039246791/menu-sync

Anyone with the link can install it. They'll see empty dropdowns unless they have a Menu Hub login that can read your Supabase data.

Updating the published version

1. Edit `menu-sync-plugin/code.js` or `ui.html` locally
2. Reload the dev copy in Figma (Plugins → Development → Menu Sync) to test
3. From any Figma file → right-click your dev plugin → **Publish new version**
4. Figma re-reviews (usually under a day on re-submits). Users get the update automatically

Private to an org instead

Requires Figma Organization or Enterprise plan (Professional doesn't have it). Once on Org, publish privately to your org — only people in the org see it, and they get updates instantly without Community review.

! Quirks of the plugin runtime.

Figma's QuickJS doesn't support `??` or `?..`. Use ternaries and guard blocks instead. Page switching also requires `figma.setCurrentPageAsync()`, not the synchronous assignment.

12 PDF / SVG export

From the plugin

- **Export PDF (live text)** button — uses Figma's native `exportAsync({format:'PDF'})` . Live text survives *only when the fonts are properly embedded*; Adobe Fonts / variable fonts get outlined
- **Export Menus tab** — bulk export every synced menu on the page as PDF or SVG

Known limitation

The CRSSD template uses Acumin Variable Concept which Figma outlines on PDF export. Workarounds: (a) switch to a non-variable font, (b) export as SVG (preserves text), (c) accept the outline and let Illustrator handle it (text is still visually crisp, just not editable).

13 CSV import / export

Column shape (canonical)

SECTION	TITLE	VT	VE	GF	DESCRIPTION	2 SIZES	SIZE	PRICE	SIZE 2	PRICE 2	STATUS	NOTES
---------	-------	----	----	----	-------------	------------	------	-------	-----------	------------	--------	-------

- **VT/VE/GF/2 Sizes:** `TRUE` / `FALSE`
- **Price columns:** any of `12` , `$12` , `$12.00` — normalized on import
- **Status:** `Active` , `Pending Approval` , `Not Added` , `Draft` (also accepts legacy `Added`)

Master Google Sheet

Apps Script (in `sheets-script/Code.gs`) provides one tab per menu with checkboxes, validation, color-coded status, and an auto-flag trigger. See section 14.

14 Master Google Sheet (Apps Script)

Lives at `sheets-script/Code.gs` in the repo. Setup guide: `sheets-script/SETUP-GUIDE.md`.

First-time setup per sheet

1. Open the Google Sheet → Extensions → Apps Script
2. Paste entire `Code.gs` contents → Save → reload spreadsheet
3. New ≡ **Menu Sync** menu appears in the sheet's menu bar
4. ≡ **Menu Sync** → **Setup VERSION LOG tab** (once)
5. ≡ **Menu Sync** → **Setup all tabs at once** (formats every non-system tab)

Workflow

- Edit items in the sheet → install the **onEditInstallable** trigger so edits auto-flag as Pending Approval
- ≡ **Menu Sync** → **Approve selected rows** to bulk-approve
- ≡ **Menu Sync** → **Email pending approvals summary** for a daily roll-up to sean@crssd.com

The Figma plugin doesn't read the sheet directly — it reads the Supabase data the app pushes. The sheet is a parallel editing surface for people who prefer spreadsheets.

15 Duplicate menus / events / series

Every level has a `...` dropdown with Edit · Duplicate · Delete (Delete admin-only):

LEVEL	WHERE	CASCADES TO
Menu	Event page → menu card	Items + same-event sponsor toggles
Event	Series page → event row	All menus + their items + sponsor toggles
Series	Brand page → series card	All events → all menus → all items

Target picker

Each duplicate modal has a cascading Brand → Series → Event picker (only the levels above the entity you're duplicating). Each select has a **+ Add new {level}...** option that inline-creates a row and immediately selects it. So you can clone a series into a brand-new brand without leaving the modal.

Never copied

- Sync metadata: `last_synced_at` , `last_sync_digest` , `preview_image_url`
- Sponsor sign-off: `sponsor_approved_at` , `sponsor_approved_by`
- Figma links: `figma_file_url` , `figma_page_name` on events
- Event dates are explicitly cleared so each new event gets its own
- Edit log entries

16 Supabase project

FIELD	VALUE
Project name	fc-events
Reference ID	wysvknamfxtbehwetjxf
URL	https://wysvknamfxtbehwetjxf.supabase.co
Region	us-west-1

Storage buckets

- `series-assets` — header logos, footer images, fonts, diet icons (per-series)
- `sponsor-logos` — uploaded sponsor SVGs (after normalization)
- `menu-previews` — PNGs uploaded by the Menu Sync plugin after every sync (used as the in-app preview thumbnail)

Key RPCs

- `log_menu_item_edit` — called from the item edit form for each changed field
- `log_sponsor_change` — called when sponsor toggles save
- `create_notification` — used by the tag selector to insert into the recipient's inbox
- `list_taggable_users` — SECURITY DEFINER function that returns admin+internal users (bypasses RLS recursion)
- `mark_menu_synced` — plugin calls after sync (writes `last_synced_at` + `preview_image_url`)
- `update_menu_preview` — plugin's preview-only refresh button
- `prune_old_notifications` — deletes archived notifications older than 30 days

Triggers

- `handle_new_user` — auto-creates a `user_profiles` row on auth signup
- `trg_notify_edit_resolved` — fires on `menu_items.edit_status` updates → notifies the editor

17 Required SQL migrations

Everything that needs to be in your Supabase project lives in `menu-hub/supabase/schema.sql`. When you spin up a fresh project, run the whole file. When iterating, you'll usually only need the new bits at the end.

The biggies that have to be in place

- Tables: brands · series · events · menus · menu_items · menu_sponsors · edit_log · user_profiles · notifications · sponsors · series_sponsors · event_sponsors · event_templates · favorites · approvers
- RLS enabled on every table with role-based policies (admin / internal / external)
- The `handle_new_user` trigger on `auth.users`
- The `trg_notify_edit_resolved` trigger on `menu_items`
- Storage buckets (above) with public-read + per-role write policies

Adding a new column

```
alter table menus add column if not exists new_field text;
```

Always use `if not exists` so re-running `schema.sql` is safe.

Adding a new RPC

```
create or replace function my_helper(p_input uuid)
returns void as $$
begin
  -- ...
end;
$$ language plpgsql security definer;
```

Use **security definer** when you need to bypass the caller's RLS (e.g. for the role check that reads `user_profiles`).

18 Deploys & rebuilds

App (Netlify)

Push to `main` → Netlify auto-builds in ~2 min → deploys to `fcmenus.netlify.app`. Build runs the `prebuild` script first (syncs `docs/walkthrough.*` → `public/walkthrough.*`).

Walkthrough docs

Edit `menu-hub/docs/walkthrough.html` (user) or `admin-walkthrough.html` (this doc). Regenerate the PDF with `npm run walkthrough:pdf` (uses headless Chrome on your Mac). Commit + push, Netlify copies into `public/` on the next build.

Plugin (dev)

Local: edit `menu-sync-plugin/code.js` or `ui.html` → reload via Figma's **Plugins** → **Development** → **Menu Sync**. No build step needed.

Plugin (published)

Right-click your dev plugin in Figma → **Publish new version**. Re-submits to Figma review. Approved users get the update automatically.

19 Troubleshooting

"Access Pending" shown to a user who used to work

Their profile fetch returned null. Usually a botched RLS policy. The fix that worked before: `SELECT * FROM user_profiles WHERE id = '{their auth uid}'` in Supabase SQL editor. If the row is gone, recreate it. If it's there, check policies on `user_profiles`.

Plugin dropdown is empty

The plugin uses an anon key, so it can read what RLS allows the unauthenticated reader. Check the policies on `brands`, `series`, `events`, `menus` — anon must have SELECT.

Plugin throws "Cannot call with documentAccess: dynamic-page"

You're using a synchronous Figma plugin API call that's blocked in dynamic-page mode (page switching, async font loads). Wrap it in the Async variant.

Sync needed chip never clears

The plugin's RPC to update `last_synced_at` failed. Check the browser console in the plugin's UI (Figma → Plugins → Development → Open Console).

Items disappeared from a menu

Probably set to Draft or Not Added accidentally. Edit Log → look for status changes. Or use the Items table's status filter (column header).

Notifications not arriving

Three places to check, in order:

1. The recipient's role — only admin + internal can be tagged (RLS limits the user list)
2. The cascade — confirm they're toggled on at Brand/Series/Event/Menu Approvals OR explicitly tagged on the edit
3. The notifications table — `SELECT * FROM notifications WHERE user_id = '{recipient uid}' ORDER BY created_at DESC LIMIT 5` — if rows are there but the inbox is empty, it's an RLS read issue

Figma sync hangs on "Generating preview image..."

The PNG upload to `menu-previews` bucket is slow on huge frames. The sync itself succeeded — the preview upload is non-blocking. Wait or cancel and re-sync.